CS571 Programming Languages

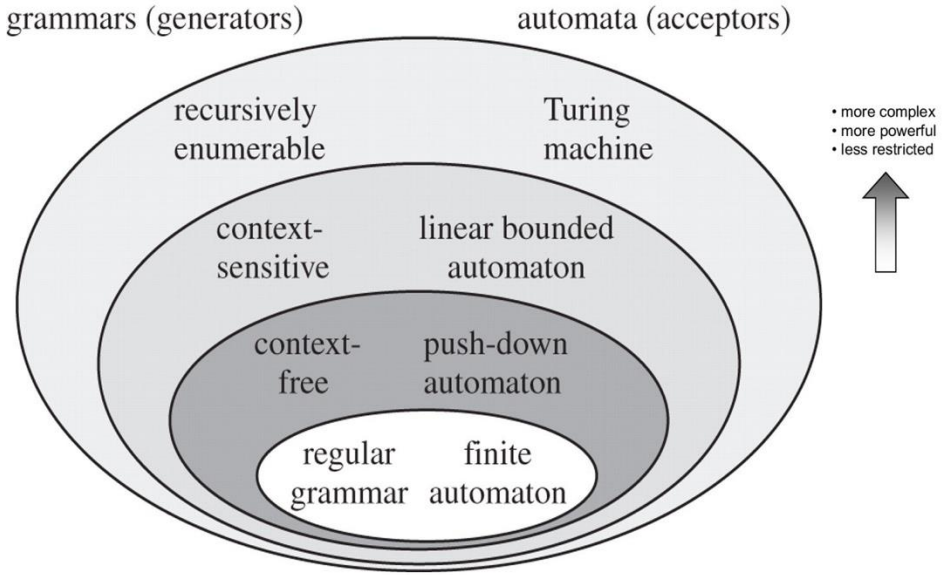
Spring 2026

3. Syntax: parsing

Agenda

1. Theory of lexing: regular expressions
2. How to build a lexer: finite automata
3. Theory of Parsing: Context-free Grammars
4. How to build a parser: recursive descent parsing

Formal Languages



The traditional Chomsky Hierarchy

Credit: Devopedia

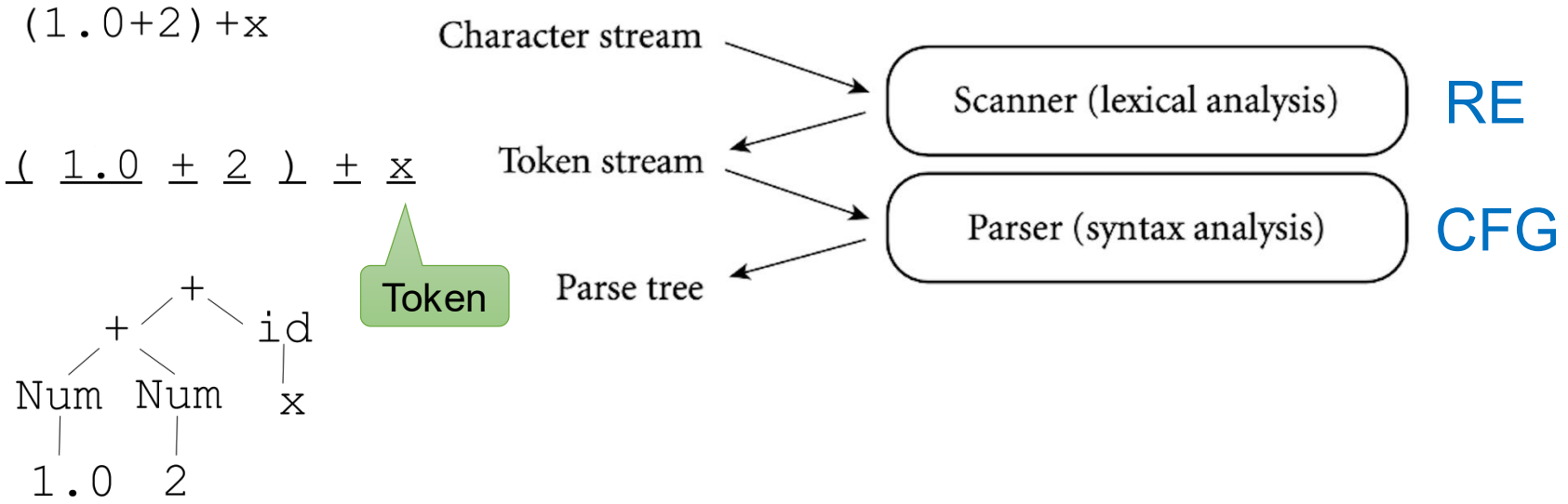
Bits with equal # of 0's and 1's

Not a regular language!

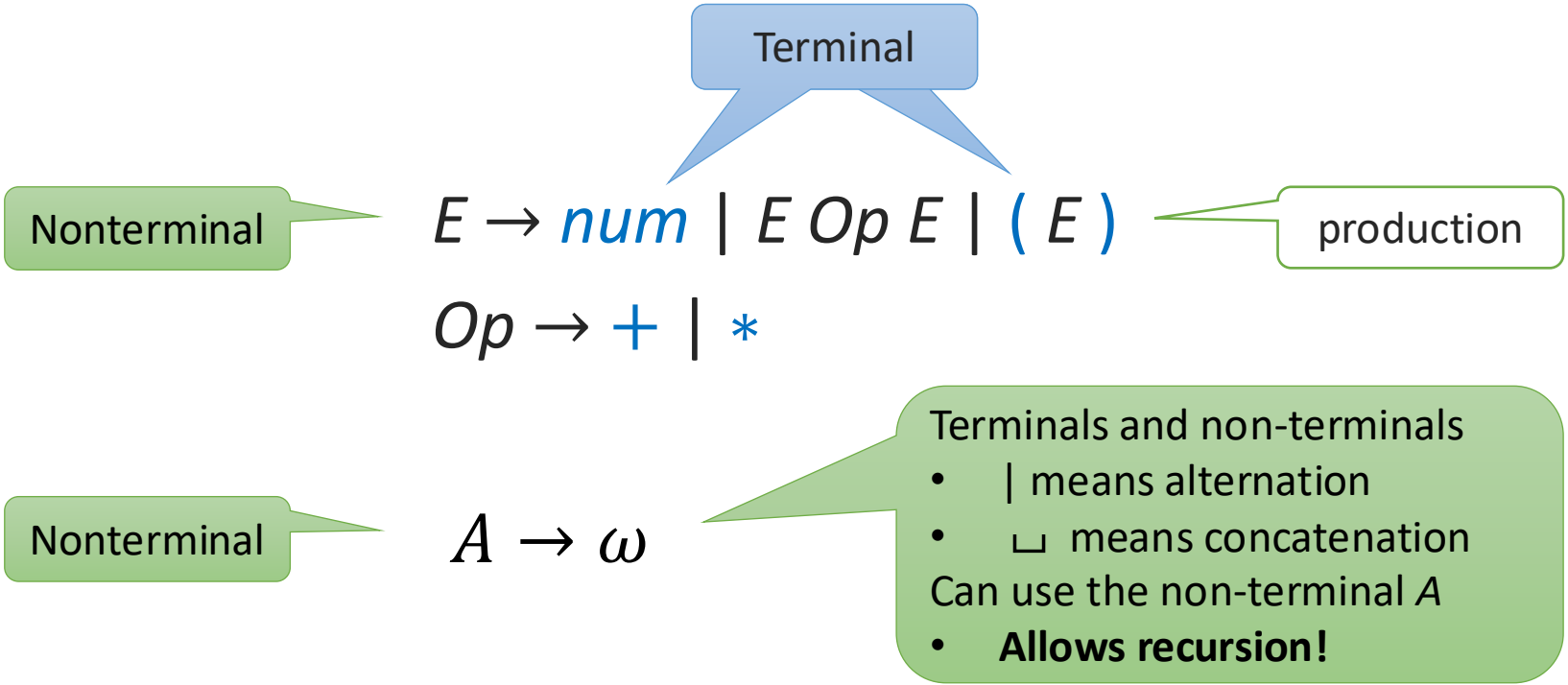
Regex Not Enough to Specify Syntax

- Regular expressions cannot be used to recognize constructs nested to **arbitrary** depths, like the language of nested parentheses. Hence, they cannot be used to specify the nested constructs used in typical programming languages
- Syntax is specified using **Context Free Grammars** (CFGs). Direct or indirect recursion in CFG's allow specifying constructs which are nested to an arbitrary depth

Typical Use of RE and CFG



CFG Example


Terminal
Nonterminal
$E \rightarrow num \mid E\ Op\ E \mid (\ E\ )$
production
$Op \rightarrow + \mid *$
Nonterminal
$A \rightarrow \omega$
Terminals and non-terminals
• | means alternation
• ␣ means concatenation
Can use the non-terminal A
• Allows recursion!

CFG Example

$E \rightarrow num \mid E\ Op\ E \mid (\ E\ )$

$Op \rightarrow + \mid *$

$E \rightarrow num$

$E \rightarrow E\ Op\ E$

$E \rightarrow (\ E\ )$

$Op \rightarrow +$

$Op \rightarrow *$

Context-Free Grammar (CFG)

A **CFG** consists of a quadruple (T, N, R, S) where:

- T is a set of *terminal* symbols (tokens)
- N is a set of *non-terminal* symbols with $T \cap N = \emptyset$
- $S \in N$ is a distinguished *start symbol*
- R is a set of *production rules*
  - Pairs $(n, rhs) \in N \times Seq(T \cup N)$, usually written $n \to rhs$
  - The first element of the pair is called the Left Hand Side (LHS)
  - The second element is called the Right Hand Side (RHS)

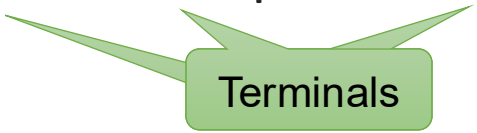
Context-Free Grammar (CFG)

Terminals

- Building block of CFG
- E.g., letters, digits, or tokens from scanner

Non-terminals

- Symbols defined by production rules
- E.g., if-stmt → IF LPAREN expr RPAREN stmt



Define Tokens by Scanner

expr → id | number | – *expr* | (*expr*) | *expr* op *expr*

where id, number, op are tokens recognized by the scanner

Building A CFG

All strings with n 0's followed by n 1's, $n \geq 1$?

$$S \rightarrow 0S1 \mid 01$$

Building A CFG

All strings with n 0's and n 1's, $n \geq 0$?

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$$

RE vs. CFG

CFG is **strictly more expressive** than RE

1. For any RE R, there is CFG C, such that $L(R) = L(C)$
2. Exists CFG C, such that for all RE R, $L(R) \neq L(C)$

Proof:

1. Proof by induction
2. See previous example

RE vs. CFG

RE	CFG
ε	$S \rightarrow \varepsilon$
a	$S \rightarrow$ a
$R_1\ R_2$	$S \rightarrow R_1\ R_2$
$R_1 \mid R_2$	$S \rightarrow R_1 \mid R_2$
$R^\star$	$S \rightarrow S\ R \mid \varepsilon$

where R_1, R_2 and R are the equivalent non-terminals for R_1, R_2 and R in RE respectively (inductive hypothesis)

Context-Free Language

$L(G)$: the language (set of strings) defined by G

Definition:

$L(G)$ is the set of all terminal strings *derivable* from the *start symbol* of G

Derivation

A sequence from *start symbol*, each step a *non-terminal* is replaced by RHS of a *production*

Derivation

A derivation of “xyz”:

var ⇒ letter var
⇒ letter letter var
⇒ letter letter letter
⇒ x letter letter
⇒ x y letter
⇒ x y z

var	→	letter var | letter
letter	→	a|b|...|Z

A multi-step reduction var ⇒* xyz

Derivation

A derivation of “x+3*y”:

```
expr  → id | number |− expr
        | (expr)| expr op expr
op    → +| −| * | /
...
```

Derivation

A derivation of “x+3*y”:

$$
\begin{aligned}
\text{expr} &\rightarrow \text{id} \mid \text{number} \mid - \text{expr} \\
&\quad \mid (\text{expr}) \mid \text{expr op expr} \\
\text{op} &\rightarrow + \mid - \mid * \mid / \\
&\ldots
\end{aligned}
$$

$$
\begin{aligned}
\text{expr} &\Rightarrow \text{expr op expr} \Rightarrow \text{id op expr} \Rightarrow \text{id op expr op expr} \\
&\Rightarrow \text{id op number op expr} \Rightarrow \text{id op number op id} \\
&\Rightarrow \text{x op number op id} \Rightarrow \text{x} + \text{number op id} \\
&\Rightarrow \cdots \Rightarrow \text{x} + 3 * \text{y}
\end{aligned}
$$

A reduction $\text{expr} \Rightarrow^{*} \text{x} + 3 * \text{y}$

Order of Derivation

Derivation can follow any order

var	→ letter var
	| letter
letter	→ a | b | ... | Z

Leftmost derivation of “xyz”:
Always replace the left most non-terminal

Order of Derivation

Derivation can follow any order

var	→ letter var
	| letter
letter	→ a | b | … | Z

Leftmost derivation of “xyz”:

Always replace the left most non-terminal

var ⇒ letter var
⇒ x var ⇒ x letter var
⇒ x y var ⇒ x y letter ⇒ x y z

Order of Derivation

Derivation can follow any order

var	→ letter var
	| letter
letter	→ a | b | ... | Z

Rightmost derivation of “xyz”:
Always replace the rightmost non-terminal

Order of Derivation

Derivation can follow any order

var	→ letter var
	| letter
letter	→ a | b | ... | Z

Rightmost derivation of "xyz":

Always replace the rightmost non-terminal

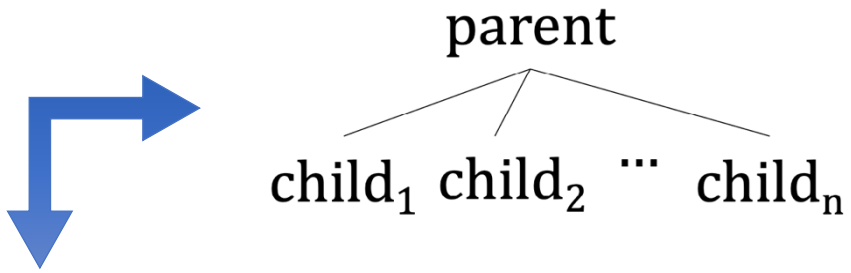
Writing derivation can be tedious

$$\begin{aligned} \text{var} &\Rightarrow \text{letter var} \\ &\Rightarrow \text{letter letter var} \Rightarrow \text{letter letter letter} \\ &\Rightarrow \text{letter letter z} \Rightarrow \text{letter y z} \Rightarrow \text{x y z} \end{aligned}$$

Parse Tree (Concrete Syntax Tree)

Derivation in graphical form

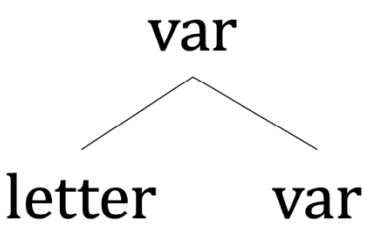
- Every node corresponds to a production rule in the grammar
- Root: start symbol
- Leaf: terminals



$$\text{parent} \Rightarrow \text{child}_1\ \text{child}_2\ \ldots \text{child}_n$$

Parse Tree

Derivation in graphical form

Derivation Step	Tree
var ⇒ letter var	

Parse Tree Example

Leftmost derivation of “xyz”

var ⇒ letter var

⇒ x var ⇒ x letter var

⇒ x y var ⇒ x y letter ⇒ x y z

var	→	letter var
		| letter
letter	→	a | b | … | Z

Parse Tree Example

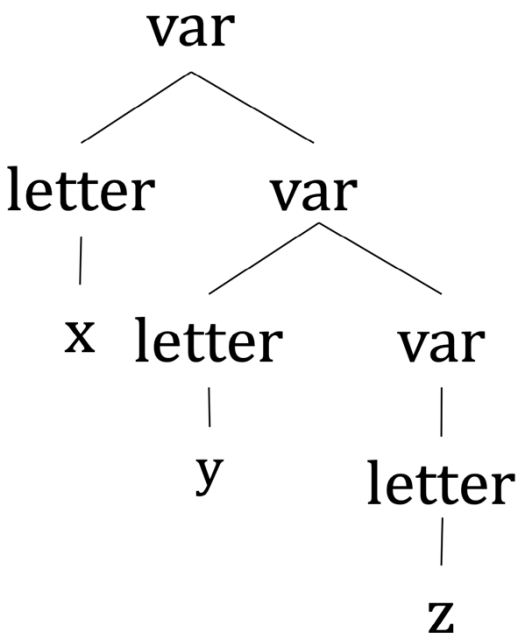
Leftmost derivation of “xyz”

var ⇒ letter var

⇒ x var ⇒ x letter var

⇒ x y var ⇒ x y letter ⇒ x y z

```
var    → letter var
         | letter
letter → a | b | ... | Z
```



Parse Tree Example

Rightmost derivation of "xyz"

var ⇒ letter var

⇒ letter letter var ⇒ letter letter letter

⇒ letter letter z ⇒ letter y z ⇒ x y z

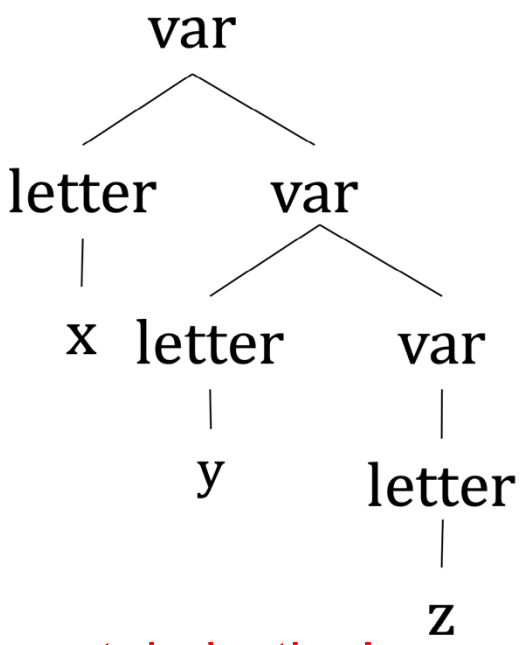
```
var     → letter var
            | letter
letter  → a | b | ... | Z
```

Parse Tree Example

Rightmost derivation of "xyz"

var ⇒ letter var
⇒ letter letter var ⇒ letter letter letter
⇒ letter letter z ⇒ letter y z ⇒ x y z

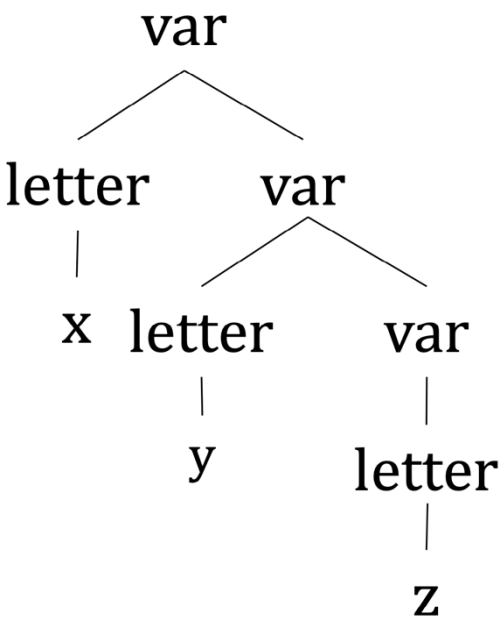
var	→ letter var
	| letter
letter	→ a | b | ... | Z



The parse tree is the same as the one from leftmost derivation!

Parse Tree Features

- Root is the start symbol
- Leaves are terminals
- Other nodes are non-terminals
- Leaves (left to right) form the parsed string



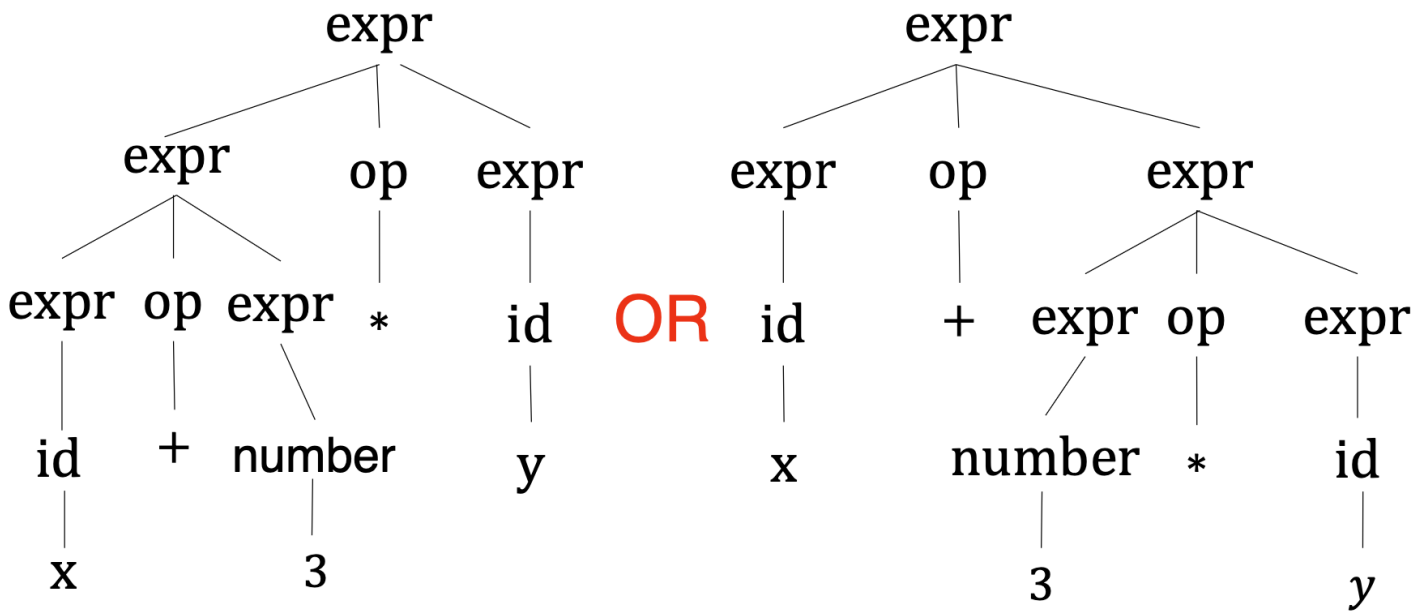
Parse Tree Example

Parse tree for “x+3*y”?

```
expr  → id | number |− expr
        | (expr)| expr op expr
op    → +| −| ∗ | /
...
```

Parse Tree Example

Parse tree for “x+3*y”?



Ambiguity

A grammar is *ambiguous* if its language contains at least one string with two or more parse trees

- Is the following language ambiguous?

$$\begin{aligned} \text{expr} &\rightarrow \text{id} \mid \text{number} \mid - \text{expr} \\ &\quad \mid (\text{expr}) \mid \text{expr op expr} \\ \text{op} &\rightarrow + \mid - \mid * \mid / \end{aligned}$$

- Yes, two parse trees for string “x+3*y”

Ambiguity

- Ambiguous grammars are not useful for specifying the concrete syntax of programming languages
- Transform grammar to remove ambiguity; alternatively, some parsers allow specifying disambiguation rules
- The problem of deciding whether a given grammar is ambiguous is **undecidable**
  - So, you will have to figure it out for yourselves!

Grammar for Math Expressions

- Precedence: which operator should be evaluated sooner
- Associativity: (binary) operator with same precedence evaluated left-to-right or right-to-left
  - Arithmetic:
    - * and / have higher precedence than + and −
    - All operators are left-associative

Example

$$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{num}$$

- Two parse trees for $1 + 2 * 3$

- Problem: can split at both $+$ and $*$

Example

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{expr} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{term} \mid \text{num} \end{aligned}$$

- Fix: force split at + (lower precedence) first

Example

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{expr} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{term} \mid \text{num} \end{aligned}$$

- Two parse trees for $1 + 2 + 3$

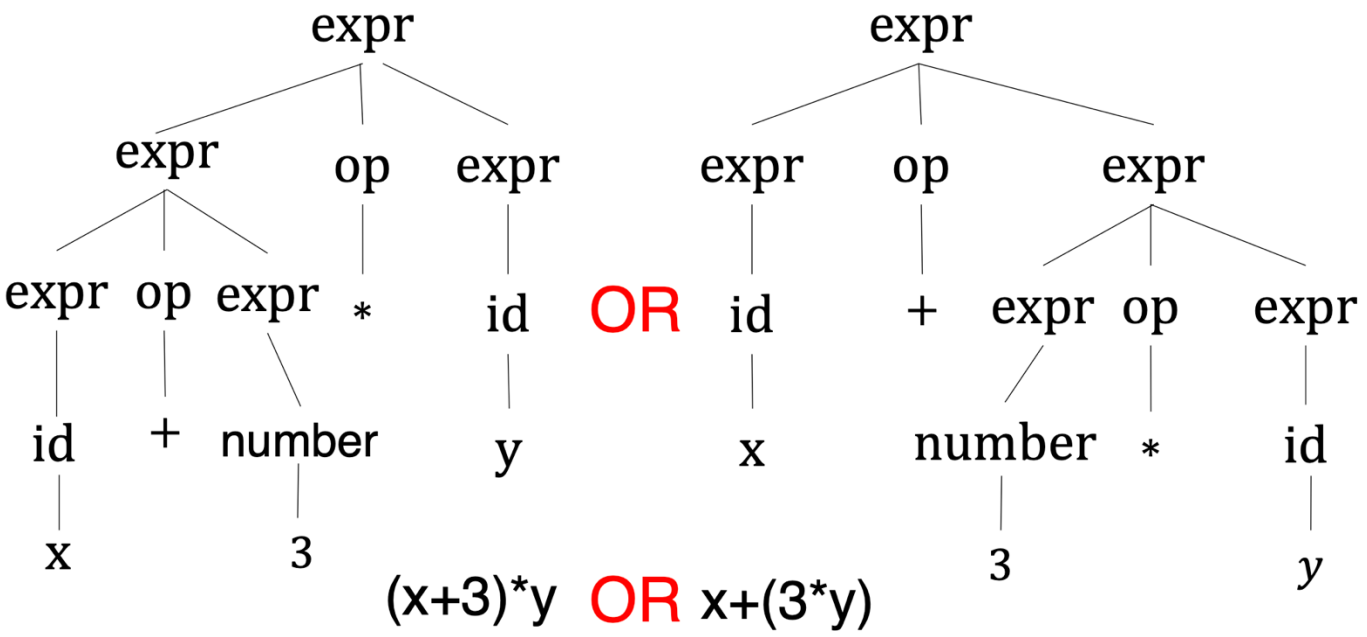
- Problem: can split at both the 1^{st} $+$ and the 2^{nd} $+$

Example

$$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{num} \mid \text{num}$$

- Fix: force split at + from right to left (left associative)

Precedence



The lower an operation is, the higher precedence it has.

Defining Precedence in Grammar

Define operations at different “levels”

$$\begin{aligned} expr &\rightarrow id \mid number \mid - expr \\ &\quad \mid (expr) \mid expr\ op\ expr \\ op &\rightarrow + \mid - \mid * \mid / \end{aligned}$$

$$expr \rightarrow expr + expr \mid expr - expr \mid term$$ Level 1

$$term \rightarrow term * term \mid term/term \mid factor$$ Level 2

$$factor \rightarrow id \mid number \mid (expr) \mid - factor$$ Level 3

The farther from start symbol, the higher precedence

Defining Precedence in Grammar

$$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} - \text{expr} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{term} \mid \text{term} / \text{term} \mid \text{factor}$$
$$\text{factor} \rightarrow \text{id} \mid \text{number} \mid (\text{expr}) \mid - \text{factor}$$

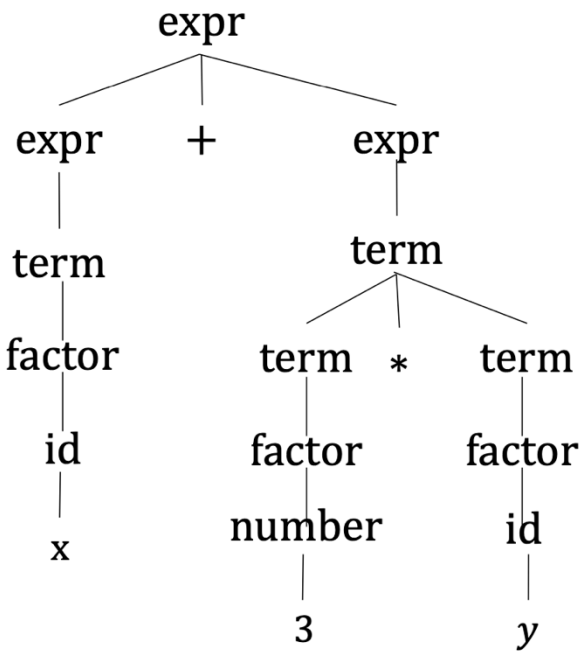
Parse tree for “x+3*y”?

Defining Precedence in Grammar

$$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} - \text{expr} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{term} \mid \text{term}/\text{term} \mid \text{factor}$$
$$\text{factor} \rightarrow \text{id} \mid \text{number} \mid (\text{expr}) \mid - \text{factor}$$

Parse tree for “x+3*y”?

Only one parse tree!



Associativity of Binary Operators

- A property of **binary** operations

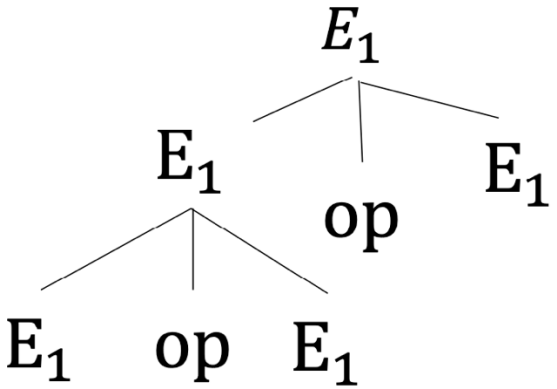
- An operator with left (right) associativity is evaluated left-to-right (right-to-left)
  - Left: +,−,*,/
  - Right: = in C: a = b = c same as a = (b = c)

Associativity in CFG

Left-recursion: LHS is the start of RHS in a derivation

$$E_1 \rightarrow^* E_1 \text{ op } \ldots$$

Left-recursion define left-associativity on op

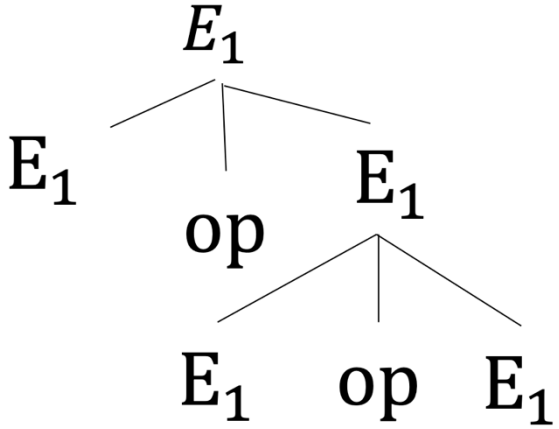


Associativity in CFG

Right-recursion: LHS is the <u>end</u> of RHS in a derivation

$$E_1 \rightarrow^* \ldots \text{op } E_1$$

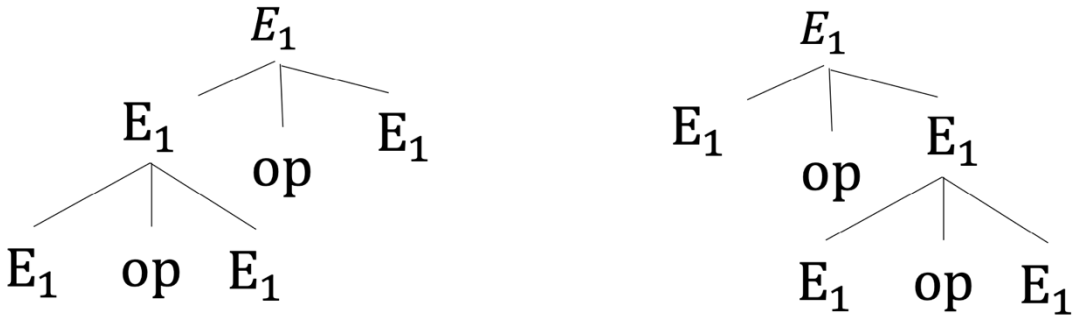
Right-recursion define right-associativity on op



Recursion and Ambiguity

Left-recursion: $E_1 \rightarrow^* E_1$ op …
Right-recursion: $E_1 \rightarrow^*$ … op E_1

A grammar with at least one operation that is <u>both</u> left and right recursive is ambiguous



Two parse trees for E_1 op E_1 op E_1

Direct Recursion in Grammar

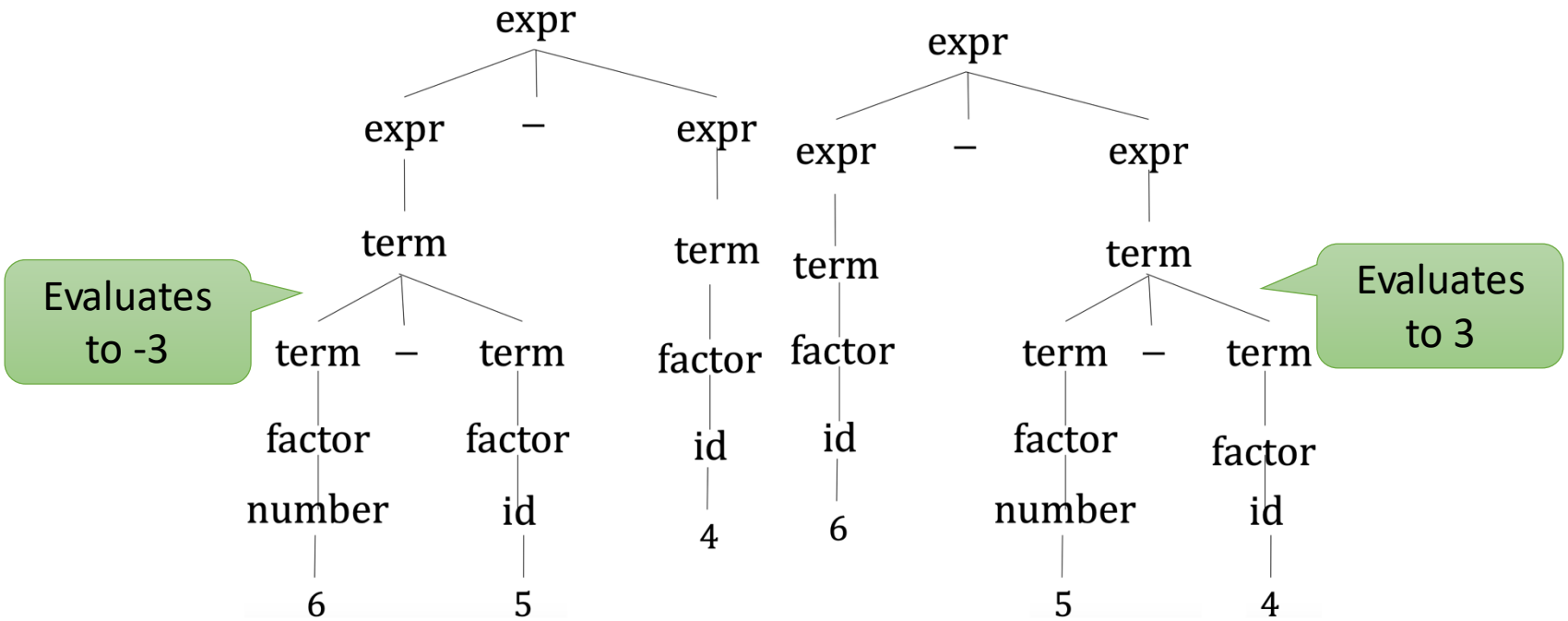
Left-recursion: $E_1 \rightarrow^* E_1$ op …
Right-recursion: $E_1 \rightarrow^*$ … op E_1

$$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} - \text{expr} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{term} \mid \text{term}/\text{term} \mid \text{factor}$$
$$\text{factor} \rightarrow \text{id} \mid \text{number} \mid (\text{expr}) \mid - \text{factor}$$

The production rule of $+, -, *, /$ is both left- and right-recursive.
So, the grammar is ambiguous.

Example

Different parse trees for string “6-5-4”



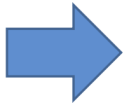
Indirect Recursion in Grammar

Left-recursion: $E_1 \rightarrow^* E_1$ op …
Right-recursion: $E_1 \rightarrow^*$ … op E_1

$$\text{equals} \rightarrow \text{var} = \text{equals1}$$

$$\text{equals1} \rightarrow \text{equals} \mid \text{var}$$

Indirect recursion: $\text{equals} \rightarrow^* \text{var} = \text{equals}$



= is
right-associative

Example

expr → expr – expr | num

- Problem: this CFG is both left and right recursive, thus $6 - 5 - 4$ has two parse trees
- Fix: remove one recursiveness

Example

expr → expr – num | num

- This CFG is left recursive, thus – is left associative
- $6 - 5 - 4$ is parsed as $(6 - 5) - 4$

Example

expr → num – expr | num

- This CFG is right recursive, thus – is right associative
- $6 - 5 - 4$ is parsed as $6 - (5 - 4)$

Defining Associativity in Grammar

$$expr \rightarrow expr + expr \mid expr - expr \mid term$$
$$term \rightarrow term * term \mid term/term \mid factor$$
$$factor \rightarrow id \mid number \mid (expr) \mid - factor$$

Remove right-recursion on $+, -, *, /$

$$expr \rightarrow expr + term \mid expr - term \mid term$$
$$term \rightarrow term * factor \mid term/factor \mid factor$$
$$factor \rightarrow id \mid number \mid (expr) \mid - factor$$

In this grammar: $+, -, *, /$ are left-associative

Example

$$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{factor} \mid \text{term}/\text{factor} \mid \text{factor}$$
$$\text{factor} \rightarrow \text{id} \mid \text{number} \mid (\text{expr}) \mid - \text{factor}$$

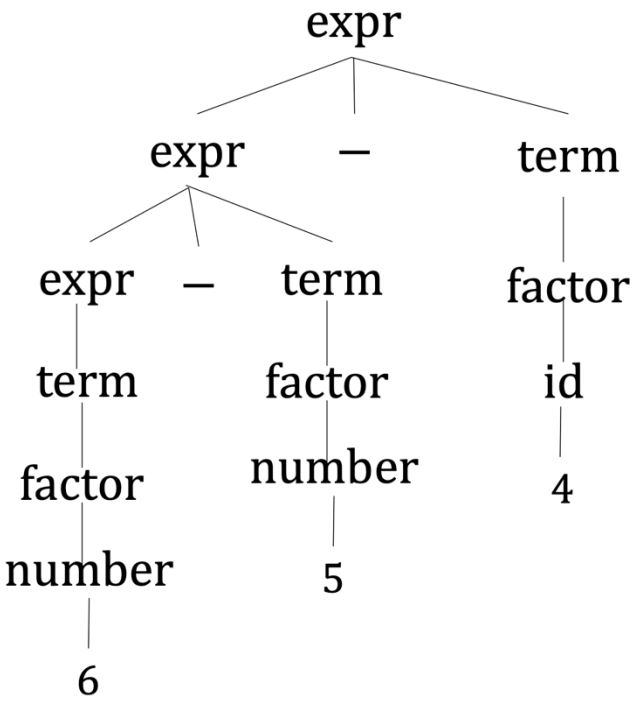
Parse tree for “6-5-4”?

Example

$$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{factor} \mid \text{term}/\text{factor} \mid \text{factor}$$
$$\text{factor} \rightarrow \text{id} \mid \text{number} \mid (\text{expr}) \mid - \text{factor}$$

Parse tree for “6-5-4”?

Only one parse tree!



Grammar for Math Expressions

- To avoid ambiguity in math expressions
  - Define precedence (different levels of operators)
  - Define associativity (left-recursion vs. right-recursion)

- Remember, avoiding ambiguity in general is undecidable!

Dangling Else Ambiguity

Given the if-statement:

if ... then ... if ... then ... else ...

Should it be interpreted as

if ... then ... (if ... then ... else ...)

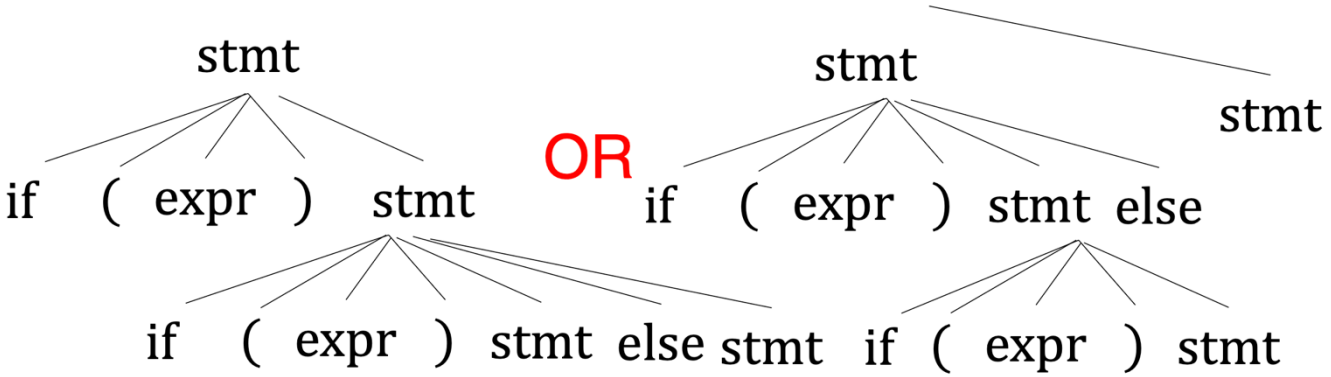
or

if ... then ... (if ... then ...) else ...

Dangling Else Ambiguity

Grammar: stmt → if expr stmt | if expr stmt else stmt

```
if (b1)                    if (b1)
  if (b2) c1       OR        if (b2) c1
  else c2                  else c2
```



Avoid Dangling Else

Approach 1: change syntax (ALGOL, Ada)

Grammar: stmt→ if expr stmt fi | if expr stmt else stmt fi

```
if (b1)
  if (b2) c1
  else c2
  fi
fi
```

```
if (b1)
  if (b2) c1
  fi
else c2
fi
```

Avoid Dangling Else

Approach 2: keep syntax, change grammar

Grammar: stmt → matched | unmatched

matched → if (expr) matched else matched

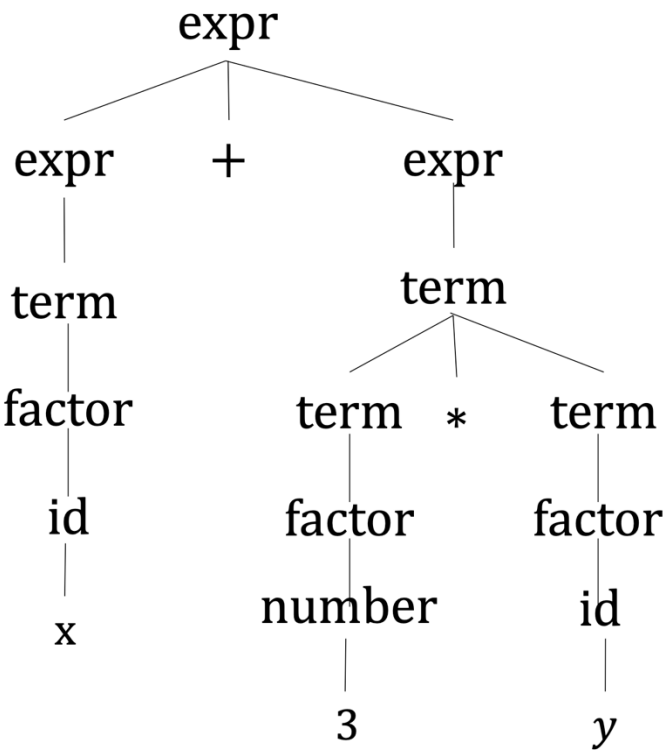
unmatched → if (expr) stmt

| if (expr) matched else unmatched

```
if (b1)
  if (b2) c1
  else c2
```

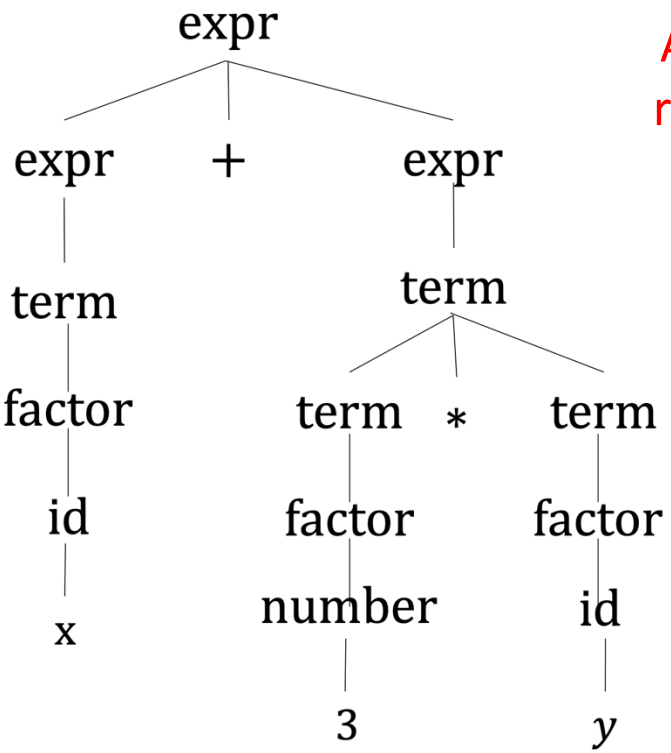
```
if (b1)
  if (b2) c1
else c2
```

Concrete Syntax Tree



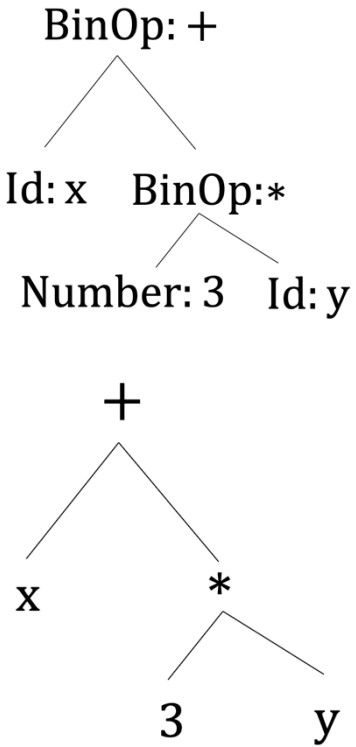
- The parse tree is very verbose.
- It tracks information only useful for parsing
  - e.g., defining precedence and associativity

Abstract Syntax Tree (AST)



AST is a compact representation of parsing tree

Or simply



Abstract Syntax Tree (AST)

- AST doesn't show the whole syntactic clutter, e.g., parentheses, non-terminals added for unambiguity
- AST keeps the same structure as parse tree
- Each node usually corresponds to an object in implementation (more manageable for later compiler stages)
- AST hides syntactical language differences

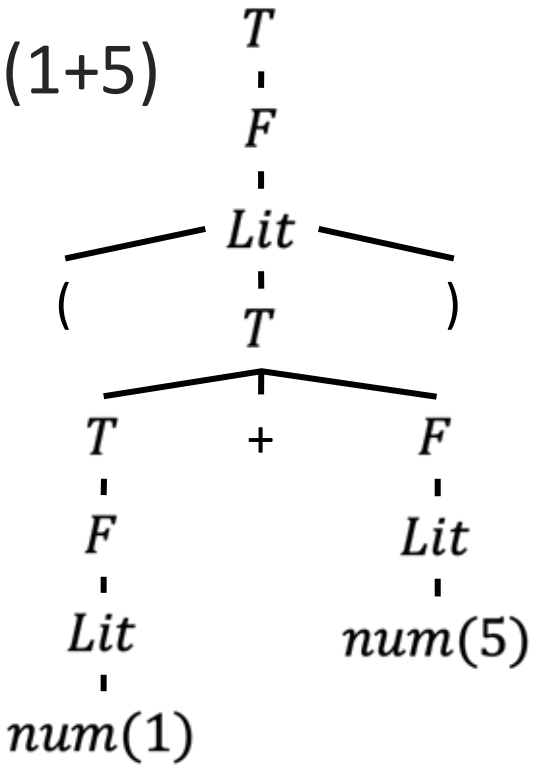
AST Example

$$E \rightarrow num \mid E\ Op\ E \mid (E)$$
$$Op \rightarrow + \mid *$$

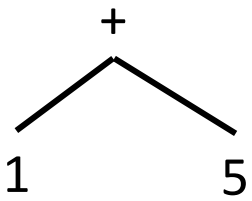
$$T \rightarrow F \mid T + F$$
$$F \rightarrow Lit \mid F * Lit$$
$$Lit \rightarrow num \mid (T)$$

AST Example

(1+5)



$$T \rightarrow F \mid T + F$$
$$F \rightarrow Lit \mid F * Lit$$
$$Lit \rightarrow num \mid (T)$$



Syntax-Directed Translation (SDT)

- SDT is defined by augmenting the CFG
  - A **translation rule** is defined for each production
- A translation rule defines the translation of the LHS nonterminal as a function of:
  - constants
  - the right-hand-side nonterminals' translations
  - the right-hand-side tokens' values

SDT Example

$T \rightarrow F$ {$$ = $1}
$T \rightarrow T + F$ {$$ = new PlusExpr($1, $3)}
$F \rightarrow Lit$ {$$ = $1}
$F \rightarrow F * Lit$ {$$ = new MulExpr($1, $3)}
$Lit \rightarrow num$ {$$ = new FloatExpr(Float.parseFloat($1.value))}
$Lit \rightarrow (T)$ {$$ = $2}

SDT Example

Grammar SDT Rules (written in a programming language or pseudocode)

Grammar	SDT Rules
$T \rightarrow F$	{$$ = $1}
$T \rightarrow T + F$	{$$ = new PlusExpr($1, $3)}
$F \rightarrow Lit$	{$$ = $1}
$F \rightarrow F * Lit$	{$$ = new MulExpr($1, $3)}
$Lit \rightarrow num$	{$$ = new FloatExpr(Float.parseFloat($1.value))}
$Lit \rightarrow (T)$	{$$ = $2}

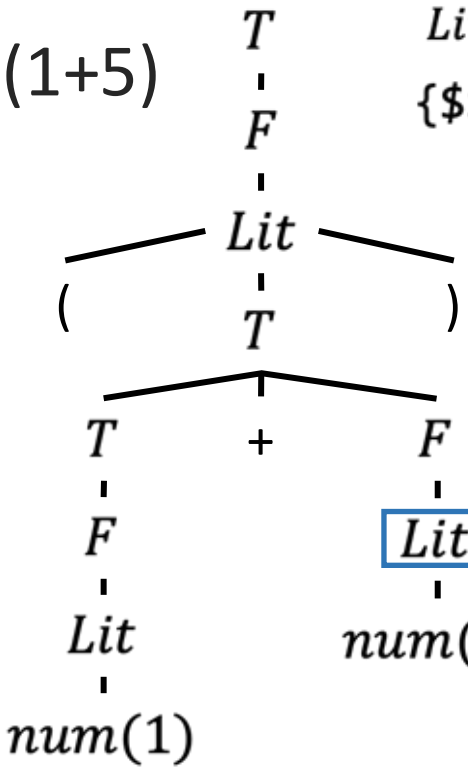
"The translation of the LHS is"

"The value of the translation of the 2nd value on the RHS"

Tokenization interface for terminals

AST Example

(1+5)

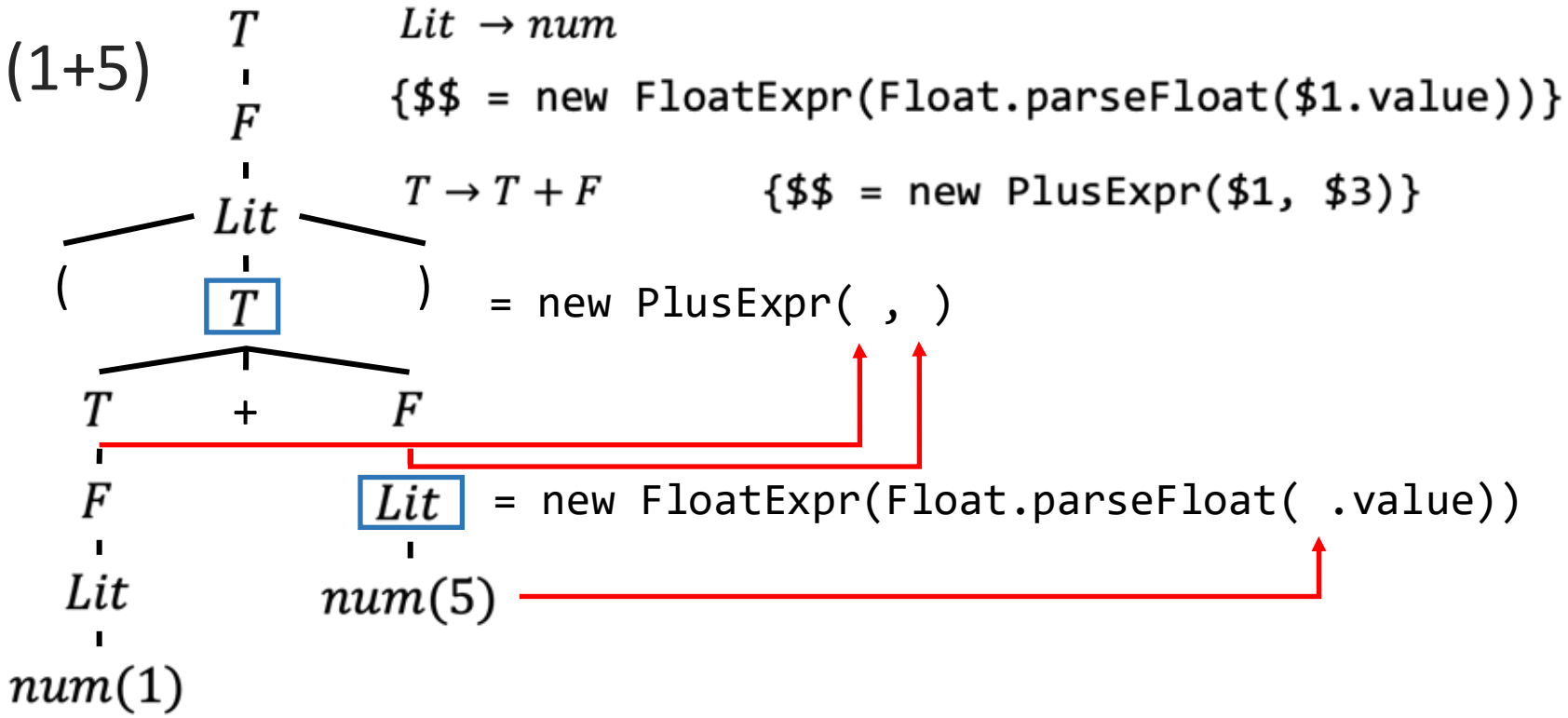


$Lit \rightarrow num$

```
{$$ = new FloatExpr(Float.parseFloat($1.value))}
```

```
= new FloatExpr(Float.parseFloat( .value))
```

AST Example


(1+5)
T
F
Lit
(T)
T + F
F
Lit
num(1)
Lit
num(5)
Lit → num
{$$ = new FloatExpr(Float.parseFloat($1.value))}
T → T + F
{$$ = new PlusExpr($1, $3)}
= new PlusExpr(,)
= new FloatExpr(Float.parseFloat(.value))

Agenda

1. Theory of lexing: regular expressions
2. How to build a lexer: finite automata
3. Theory of Parsing: Context-free Grammars
4. How to build a parser: recursive descent parsing

So you want to build a parser…

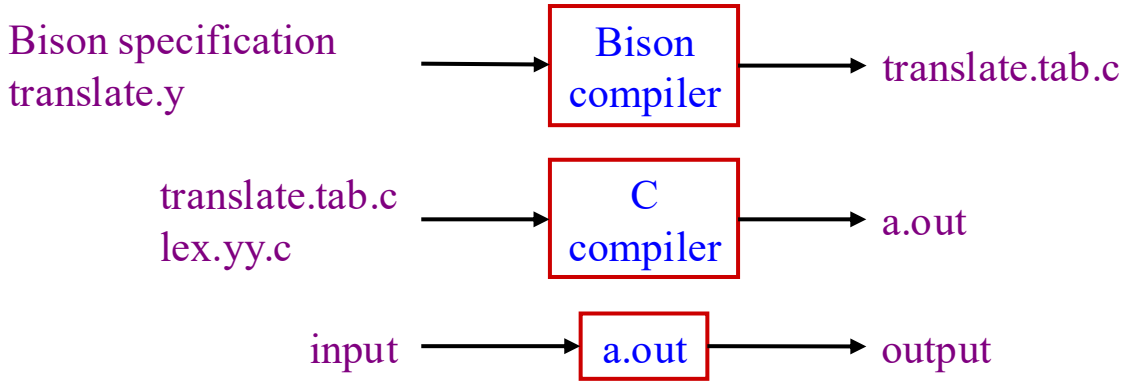
Your options:

- Recursive Descent Parsing
  - Parser combinators (e.g. Parsec)
  - Parsing Expression Grammars (PEGs)
- Parser Generator (e.g. yacc/Bison/ANTLR)
- Earley Parsing
  - Closely related: CYK parsing

Special Class of CFG

- LL(k)
  - Can be parsed with recursive descent parsing.
- LR(k)
  - Can be parsed with shift-reduce parsing
  - Shift-reduce parsing is hard to implement by hand
  - Original motivation for parser-generators
- Deterministic CFGs
  - Subsumes LL and LR

Parser Generators (Bison, YACC, ANTLR)



- **yyparse( )** implements grammar
- **yylex( )** implements the lexical analyzer
- **yyerror( )** is called by the parser to report an error
- **main( )**

Parser Generators (Bison, YACC, ANTLR)

```
stmt:
            | stmt expr_stmt
;
expr_stmt: expr TOK_SEMICOLON
                        | TOK_PRINTLN expr TOK_SEMICOLON
                      { fprintf(stdout, "the value is %d\n", $2); }
;
expr:
        expr TOK_ADD expr
         { $$ = $1 + $3;}
        | expr TOK_SUB expr
         { $$ = $1 - $3; }
        | expr TOK_MUL expr
          { $$ = $1 * $3;   }
     ……
```

Earley Parsing

- Key idea: dynamic programming
  - `fib(n) = fib(n-1) + fib(n-2)` versus
  - `fib[n] = fib[n-1] + fib[n-2]`

- Efficiency
  - $O(n^3)$ for arbitrary CFG
  - $O(n^2)$ for unambiguous CFG (which every PL CFG should be)
  - $O(n)$ for *deterministic* CFG (which include LR, LL, and LALR)

Recursive Descent Parsing

- Recursive-descent is a simple way of writing parsers manually
- A recursive-descent parser is a top-down parser which descends into derivation using a set of mutually-recursive functions
- Structure of recursive-descent parsing program mirrors CFG.
- Rather severe restrictions on class of CFG's which can be handled using this technique

Recursive Descent Parsing

- State of the parser
  - The token stream `tokens`
  - The current index `i`

- Supporting function interface
  - `peek(t,k)`, which returns true iff `tokens[i+k] == t`
  - `consume(t)`, which sets `i = i + 1` and returns true iff `tokens[i].type == t`

Recursive Descent Parsing

- Nearly automatic translation from grammar to parser

- For each nonterminal, generate a function to match that nonterminal
  - The function selects between productions using `peek`
  - For the appropriate production, execute its nonterminals and terminals in order
    - For each nonterminal, call its function
    - For each terminal `t`, call `consume(t)`

Recursive Descent Parsing

Consider following grammar for a list of comma-separated ID's terminated by a ;

$$IDList \rightarrow id\ IDListTail$$
$$IDListTail \rightarrow ,\ id\ IDListTail \mid ;$$

```
void idList() {

}
void idListTail() {

}
```

Recursive Descent Parsing

Consider following grammar for a list of comma-separated ID's terminated by a ;

$$IDList \rightarrow id\ IDListTail$$
$$IDListTail \rightarrow , \ id\ IDListTail \mid \ ;$$

```
void idList() {
  consume(ID);
  idListTail();
}
void idListTail() {




}
```

Recursive Descent Parsing

```
void idList() {
  consume(ID);
  idListTail();
}
void idListTail() {
  if (peek(‘,’), 0) {
    consume(‘,’);
    consume(ID);
    idListTail();
  } else { consume(‘;’); }
}
```

Recursive Descent Parsing: Recognizer

```
boolean idList() {
  boolean v1 = consume(ID);
  boolean v2 = idListTail(); return v1 && v2;
}
boolean idListTail() {
  if (peek(‘,’), 0) {
    boolean v1 = consume(‘,’);
    boolean v2 = consume(ID);
    boolean v3 = idListTail(); return v1 && v2 && v3;
  } else { return consume(‘;’); }
}
```

Recursive Descent Parsing: Parser

Consider following grammar for a list of comma-separated ID's terminated by a ;

$IDList \rightarrow id\ IDListTail$ { $$ = new List($1.value, $2) }

$IDListTail \rightarrow ,\ id\ IDListTail$ { $$ = new List($2.value, $3) }

$IDListTail \rightarrow ;$ { $$ = new List(); }

Recursive Descent Parsing: Parser

```
List idList() {
  Token v1 = consume(ID);
  List v2 = idListTail(); return new List(v1.value, v2);
}
List idListTail() {
  if (peek(‘,’), 0) {
    Token v1 = consume(‘,’);
    Token v2 = consume(ID);
    List v3 = idListTail(); return new List(v2.value, v3);
  } else { Token v1 = consume(‘;’); return new List(); }
}
```

SDT Rules

Recursive Descent Problems

- Consider fragment of arithmetic expression grammar:

$E \rightarrow E + T \mid T$

with following function for non-terminal expr:

```
void expr() {
  if (...) {
    expr(); consume('+'); term();
  } else { term(); }
}
```

Recursive Descent Problems

- What is the branch condition `(...)` in `if (...)`?
- `expr()` calls `expr()` directly without changing lookahead: results in an infinite loop!!
- Recursive-descent parsers cannot handle CFG's with left-recursive (direct or indirect) rules

Dealing with Left-Recursion

- Left Recursive Grammar

$$E \rightarrow E + T \mid T$$

- Parsing Expression Grammars (PEGs)

$$E \rightarrow (T +)^* T$$

- Direct left-recursion elimination via grammar transformation

$$E \rightarrow T \mid E' T$$
$$E' \rightarrow T + E'$$

- Converting to right-recursion

$$E \rightarrow T + E \mid T$$

Backtracking Recursive Descent

- So far, we have looked at LL(1) recursive descent parsing
- Most grammars for programming languages are in a class called LL(k)
  - This means they need a peek lookahead method that looks ahead up to k characters
- Grammars *not* in LL(k) require backtracking with recursive descent
- Backtracking can be exponential time

Backtracking Recursive Descent

$E \rightarrow T + E \mid T$

$T \rightarrow num$

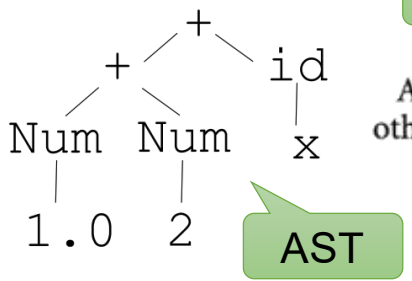
```
bool expr() {
  init = cur_state();
  if(term() && consume('+') &&
    expr()) { return true; }
  reset(init);
  if (term()) { return true; }
  else { return false; }
}
bool term() {
  return consume(NUM);
}
```

Source Code to Target Code

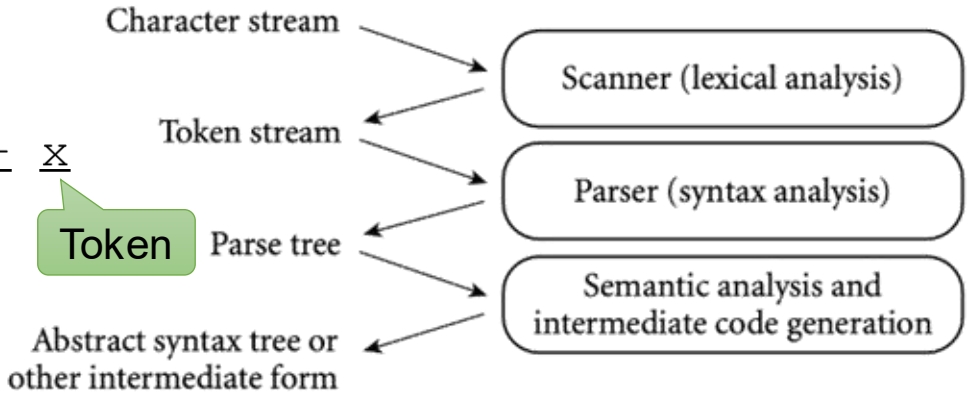
```
(1.0+2)+x
```

```
( 1.0 + 2 ) + x
```

Token



AST



RE, FSA

CFG, PDA

Type check
Scope check

`+` only takes numbers
`x` is in scope (visible)

Coming next:
Names, Scopes & Bindings